

PREFIX SUM + HASHING

Detailed DSA Pattern — Textbook Edition

Google Interview Preparation

The complete conversation converted into a structured textbook-style PDF, preserving the content and examples.

Core principle:

Prefix Sum converts a subarray-sum problem into a difference between two prefix sums, while Hashing lets us find the required previous prefix sum in $O(1)$ average time.

1. The Problem Family

The **Prefix Sum + Hashing** pattern is one of the most important array patterns for Google-style DSA interviews. It is especially powerful for **subarray-sum problems**, where brute force would require repeatedly calculating sums.

The key idea is:

Prefix Sum converts a subarray-sum problem into a difference between two prefix sums, while Hashing lets us find the required previous prefix sum in $O(1)$ average time.

You should immediately think of **Prefix Sum + Hashing** when you see questions involving:

- contiguous subarrays
- sum of a subarray
- number of subarrays satisfying a sum condition
- longest subarray satisfying a sum condition
- subarray sum equals K
- zero-sum subarrays
- equal number of certain elements
- cumulative/running sums
- finding whether a subarray satisfies a target condition

Typical wording:

"Find the number of contiguous subarrays whose sum is K."

or:

"Find the longest contiguous subarray whose sum is K."

or:

"Determine whether there exists a contiguous subarray whose sum is 0."

These are strong signals.

2. First Understand "Subarray"

A **subarray** is a contiguous portion of an array.

```
nums = [2, 4, 1, 3]
```

Examples:

```
[2]
```

```
[4]
```

```
[1]
```

```
[3]
```

```
[2, 4]
```

```
[4, 1]
```

[1, 3]

[2, 4, 1]

[4, 1, 3]

[2, 4, 1, 3]

But:

[2, 1]

is **not** a subarray because the elements are not contiguous.

This distinction is critical.

Subarray vs subsequence

Subarray:
contiguous

Subsequence:
does not necessarily need to be contiguous

Prefix Sum + Hashing is primarily a **subarray** technique.

3. Why Brute Force Is Expensive

Suppose:

nums = [1, 2, 3, 4]

You could enumerate every possible subarray.

There are:

$$\frac{n(n+1)}{2}$$

possible subarrays.

Therefore:

$$O(n^2)$$

subarrays exist.

If you calculate each subarray's sum independently, the solution can become:

$$O(n^3)$$

With a running sum inside the nested loops, you can reduce it to:

$$O(n^2)$$

But we can often do better.

4. Prefix Sum

A **prefix sum** stores the cumulative sum from the beginning of the array.

Given:

```
nums = [2, 4, 1, 3]
```

Create:

```
prefix:  
[2, 6, 7, 10]
```

Because:

```
prefix[0] = 2  
  
prefix[1] = 2 + 4 = 6  
  
prefix[2] = 2 + 4 + 1 = 7  
  
prefix[3] = 2 + 4 + 1 + 3 = 10
```

A more useful representation is to include a zero:

```
prefix = [0, 2, 6, 7, 10]
```

This makes the mathematics much cleaner.

5. The Core Formula

Suppose:

```
prefix[j] = sum(nums[0 ... j-1])
```

and:

```
prefix[i] = sum(nums[0 ... i-1])
```

Then:

```
prefix[j] - prefix[i]
```

gives:

```
sum(nums[i ... j-1])
```

Therefore:

subarray sum = current prefix - previous prefix

This is the foundation of the entire pattern.

6. Example

Consider:

```
nums = [2, 4, 1, 3]
```

Prefix:

```
index:  0   1   2   3   4  
prefix: 0   2   6   7  10
```

Suppose we want:

```
sum = 5
```

Look at:

```
prefix = 7
```

We need an earlier prefix:

$$7 - 5 = 2$$

And indeed:

```
prefix[1] = 2  
prefix[3] = 7
```

Therefore:

$$7 - 2 = 5$$

The corresponding subarray is:

```
[4, 1]
```

7. Where Hashing Enters

The formula says:

```
current_prefix - previous_prefix = K
```

Rearrange:

```
previous_prefix = current_prefix - K
```

So for every current prefix sum S , we need to ask:

Have I seen $S - K$ before?

This is exactly what a hash map can answer efficiently.

```
HashMap  
prefix_sum → information about that prefix
```

For example:

```
{  
  0: 1,  
  2: 1,  
  6: 1,  
  7: 1  
}
```

Lookup is:

```
O(1)
```

on average.

Therefore:

```
Prefix Sum  
+  
Hash Map  
=  
O(n)
```

8. The Fundamental Transformation

This is the pattern you should memorize conceptually, not mechanically.

Given:

```
sum(subarray) = K
```

Transform:

```
current_prefix - previous_prefix = K
```

into:

```
previous_prefix = current_prefix - K
```

Then:

```
For every current prefix:
    required = current_prefix - K

    check whether required exists
```

That is the entire central idea.

9. The Most Important Example — Subarray Sum Equals K

Given:

```
nums = [1, 1, 1]
K = 2
```

Expected answer:

2

The valid subarrays are:

```
[1, 1]
[1, 1]
```

10. Brute Force Approach

We could check every possible subarray.

```
count = 0

for i in range(len(nums)):
    current_sum = 0

    for j in range(i, len(nums)):
        current_sum += nums[j]

        if current_sum == k:
            count += 1
```

Complexity:

```
Time: O(n2)
Space: O(1)
```

This is already better than $O(n^3)$, but we can do $O(n)$.

11. Prefix Sum + Hashing Solution

```
def subarray_sum(nums, k):
    prefix_count = {0: 1}

    prefix_sum = 0
    count = 0

    for num in nums:
        prefix_sum += num

        required = prefix_sum - k

        if required in prefix_count:
            count += prefix_count[required]

        prefix_count[prefix_sum] = (
            prefix_count.get(prefix_sum, 0) + 1
        )

    return count
```

Complexity:

Time: $O(n)$ average
Space: $O(n)$

12. Why `{0: 1}` Is So Important

This is one of the most common interview mistakes.

We initialize:

```
prefix_count = {0: 1}
```

Why?

Because we need to represent:

Before processing any elements, we have seen a prefix sum of zero once.

Consider:

```
nums = [2, 3]
K = 5
```

After processing:

2

prefix:

2

After processing:

3

prefix:

5

Required:

$$5 - 5 = 0$$

We need to find prefix 0.

That represents the empty prefix before the array.

Therefore:

$$\text{prefix } 0 \rightarrow \text{count } 1$$

allows us to recognize:

[2, 3]

as a valid subarray.

13. Step-by-Step Dry Run

Consider:

```
nums = [1, 2, 3]
K = 3
```

Initialize:

```
prefix_sum = 0
count = 0
```

```
map = {
    0: 1
}
```

Process 1

```
prefix_sum = 1
```

Required:

$$1 - 3 = -2$$

`-2` doesn't exist.

Add:

$$1 \rightarrow 1$$

Map:

```
{
    0: 1,
    1: 1
}
```

Process 2

```
prefix_sum = 3
```

Required:

$$3 - 3 = 0$$

`0` exists once.

Therefore:

```
count = 1
```

Map:

```
{
  0: 1,
  1: 1,
  3: 1
}
```

The subarray is:

```
[1, 2]
```

Process 3

```
prefix_sum = 6
```

Required:

```
6 - 3 = 3
```

`3` exists once.

Therefore:

```
count = 2
```

The second valid subarray is:

```
[3]
```

Answer:

```
2
```

14. Why Do We Store Counts?

This is extremely important.

Consider:

```
nums = [0, 0, 0]
K = 0
```

Every possible subarray has sum zero.

There are:

$$3(3+1)/2 = 6$$

valid subarrays.

Prefix sums:

```
0
0
0
0
```

The same prefix sum occurs multiple times.

Therefore the hash map must store:

`prefix_sum → frequency`

rather than merely:

`prefix_sum → true`

15. Dry Run With Zeros

Start:

```
map = {0: 1}
count = 0
prefix = 0
```

First zero:

```
prefix = 0
required = 0
```

Map contains:

`0 → 1`

So:

```
count = 1
```

Then increment frequency:

`0 → 2`

Second zero:

```
required = 0
```

Map:

`0 → 2`

So:

```
count += 2
```

Now:

```
count = 3
```

Third zero:

```
count += 3
```

Final:

```
count = 6
```

Exactly correct.

16. The Deeper Mathematical Interpretation

Suppose prefix sums are:

`P0, P1, P2, ..., Pn`

For a subarray from *i* to *j*:

$$\text{sum}(i,j) = P(j+1) - P(i)$$

We want:

$$P(j+1) - P(i) = K$$

Therefore:

$$P(i) = P(j+1) - K$$

So when we reach a prefix sum:

S

we simply ask:

How many previous prefix sums equal $S - K$?

Each matching prefix represents one valid subarray.

This is why the algorithm works.

17. Prefix Sum + Hashing Template

For **counting subarrays with sum K**, remember this conceptual template:

```
def solve(nums, k):
    prefix_count = {0: 1}

    prefix_sum = 0
    answer = 0

    for num in nums:
        prefix_sum += num

        required = prefix_sum - k

        if required in prefix_count:
            answer += prefix_count[required]

        prefix_count[prefix_sum] = (
            prefix_count.get(prefix_sum, 0) + 1
        )

    return answer
```

Do not memorize the code blindly.

Memorize the reasoning:

```
Current prefix
↓
Need previous prefix
↓
current - previous = K
↓
previous = current - K
↓
HashMap lookup
```

18. Why Sliding Window Doesn't Always Work

This is an important Google interview distinction.

Sliding Window is extremely useful when the array has properties that allow the window to move predictably.

For example, with:

positive numbers

if the current sum is too large, shrinking the window decreases the sum.

But consider:

nums = [3, -2, 1, -1]

Negative numbers destroy that monotonic behavior.

You cannot safely say:

sum too large → move left
sum too small → move right

because adding/removing an element can increase or decrease the sum unpredictably.

Prefix Sum + Hashing handles negative numbers naturally.

19. Prefix Sum vs Sliding Window

Feature	Prefix Sum + Hashing	Sliding Window
Contiguous subarray	Yes	Yes
Negative numbers	Yes	Often problematic
Count subarrays with exact sum	Excellent	Usually not
Exact target sum	Excellent	Depends
Typical complexity	O(n)	O(n)
Extra space	O(n)	Usually O(1)
Core idea	Prefix difference	Dynamic window

Interview rule

If you see:

subarray
+
exact sum
+
negative numbers possible

strongly consider:

Prefix Sum + Hashing

20. Prefix Sum + Hashing for Zero Sum

A special case:

$K = 0$

Then:

$\text{required} = \text{prefix_sum}$

So we are looking for repeated prefix sums.

Why?

$\text{prefix}[j] = \text{prefix}[i]$

then:

$$\text{prefix}[j] - \text{prefix}[i] = 0$$

Therefore:

$\text{sum}(i \dots j-1) = 0$

Important insight

Repeated prefix sums imply a zero-sum subarray.

21. Longest Subarray With Sum K

The same prefix-sum idea can solve a different problem:

"Find the length of the longest subarray whose sum is K."

But there is a crucial difference.

For counting:

$\text{prefix} \rightarrow \text{frequency}$

For longest length:

$\text{prefix} \rightarrow \text{earliest index}$

Why earliest?

Suppose current index is j and required prefix was first seen at index i .

$$\text{length} = j - i$$

To maximize the length, we want the **earliest occurrence** of that prefix sum.

22. Example

$\text{nums} = [1, -1, 5, -2, 3]$
 $K = 3$

We maintain:

$\text{prefix_sum} \rightarrow \text{earliest_index}$

When we see:

$\text{prefix_sum} = S$

look for:

S - K

If it exists at the earliest index:

```
length = current_index - earliest_index
```

Critical rule

Do **not** overwrite the first occurrence.

```
if prefix_sum not in first_index:  
    first_index[prefix_sum] = i
```

This differs from the counting problem.

23. Two Similar Problems, Two Different Hash Maps

This distinction is interview-important.

Count subarrays

prefix → frequency

because we need:

How many previous prefixes match?

Longest subarray

prefix → earliest index

because we need:

Which previous prefix gives the maximum distance?

24. Common Prefix Sum + Hashing Variations

Variation 1

Count subarrays with sum K

Store:

prefix → count

Variation 2

Longest subarray with sum K

Store:

prefix → earliest index

Variation 3

Does a zero-sum subarray exist?

Store:

prefix → seen

Repeated prefix means zero-sum subarray.

Variation 4

Count subarrays with sum divisible by K

Now the important concept becomes:

`prefix_sum % K`

If two prefix sums have the same remainder:

$$(P_j \bmod K) = (P_i \bmod K)$$

then:

$$(P_j - P_i) \bmod K = 0$$

Therefore the subarray sum is divisible by K.

This is a natural next-level extension of the pattern.

25. Important: Modulo Variation

Suppose:

```
nums = [4, 5, 0, -2, -3, 1]
K = 5
```

Compute prefix sums and their remainders modulo 5.

The important idea is:

```
same remainder
  ↓
difference divisible by K
```

This gives another major interview pattern:

Prefix Sum + Hashing + Modulo

26. Handling Negative Modulo

In Python:

```
(-2) % 5
```

produces:

```
3
```

which is useful because Python gives a non-negative remainder for a positive divisor.

In other languages, modulo behavior may require normalization.

Conceptually:

```
remainder = ((prefix % k) + k) % k
```

is a common language-independent normalization approach.

You should understand this because it appears in interview implementations.

27. Common Mistakes

Mistake 1 — Forgetting `{0: 1}`

Incorrect:

```
prefix_count = {}
```

Correct for counting:

```
prefix_count = {0: 1}
```

Mistake 2 — Storing only `True`

Incorrect mental model:

```
seen[prefix] = True
```

for a counting problem.

You need:

```
prefix → frequency
```

because the same prefix can appear multiple times.

Mistake 3 — Updating the map too early

The conceptual order is:

1. Update prefix sum
2. Find required prefix
3. Add its frequency
4. Store current prefix

This avoids accidentally using the current prefix as its own previous prefix in situations where that would be incorrect.

28. Mistake 4 — Confusing Subarray and Subsequence

If the problem says:

```
subarray
```

think:

```
CONTIGUOUS
```

Prefix Sum naturally works with contiguous ranges.

29. Mistake 5 — Using Sliding Window Automatically

Don't see:

```
"subarray sum"
```

and immediately implement sliding window.

Ask:

Can the array contain negative numbers?

If yes, prefix sum + hashing is often the stronger candidate.

30. Mistake 6 — Using Prefix Array When You Don't Need It

You don't always need:

```
prefix = [0] * (n + 1)
```

For many one-pass problems, you only need:

```
prefix_sum
```

This reduces the implementation's memory overhead.

Instead of:

```
prefix array → O(n)
```

you can maintain:

```
current prefix → O(1)
```

while the hash map itself uses O(n).

31. Complexity

For:

```
Subarray Sum Equals K
```

Time

Each element is processed once.

Hash lookup:

```
O(1) average
```

Therefore:

O(n)

Space

The hash map can contain up to n different prefix sums.

Therefore:

O(n)

32. The Invariant

For Google interviews, don't just explain the algorithm.

Explain the **invariant**.

For the counting version:

After processing the first i elements, the hash map contains the frequency of every prefix sum encountered so far, and count contains the number of valid subarrays ending at or before the current position.

This is a strong interview-level explanation.

33. How to Recognize the Pattern

Question 1

Is it asking about a:

contiguous subarray?

If no → this pattern may not apply.

Question 2

Is there a condition involving:

sum?

If yes → consider prefix sum.

Question 3

Is there an exact target?

$sum == K$

Then consider:

$current_prefix - K$

Question 4

Do I need to repeatedly find a previous prefix?

If yes:

HashMap

Question 5

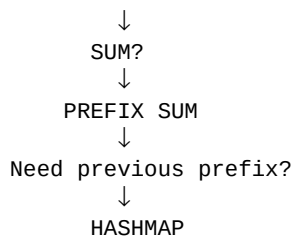
What does the map need to store?

count?
earliest index?
existence?

This determines the exact variant.

34. Pattern Recognition Cheat Sheet

CONTIGUOUS SUBARRAY



Then determine what the HashMap stores:

COUNT SUBARRAYS
 $\rightarrow \text{prefix} \rightarrow \text{frequency}$

LONGEST SUBARRAY
 $\rightarrow \text{prefix} \rightarrow \text{earliest index}$

EXISTS?
 $\rightarrow \text{prefix} \rightarrow \text{seen}$

DIVISIBLE BY K
 $\rightarrow \text{prefix} \% K \rightarrow \text{frequency}$

35. Prefix Sum Is Not Only for Sum

The broader idea is:

Convert a range operation into a difference between cumulative states.

For sums:

$\text{prefix}[j] - \text{prefix}[i]$

For other problems, we can transform the input so that a similar cumulative-state technique works.

For example, consider:

Find the longest subarray containing equal numbers of 0s and 1s.

Transform:

$0 \rightarrow -1$
 $1 \rightarrow +1$

Then the problem becomes:

Find the longest subarray with sum 0.

Now prefix sum + hashing applies.

This is a very important interview transformation.

36. Equal 0s and 1s Example

Given:

$[0, 1, 0, 1, 1, 0]$

Transform:

$[-1, +1, -1, +1, +1, -1]$

If a prefix sum repeats:

```
prefix[j] == prefix[i]
```

then:

$$\text{prefix}[j] - \text{prefix}[i] = 0$$

meaning the subarray contains equal numbers of +1 and -1.

Therefore:

equal 0s and 1s

This is a classic example of **problem transformation + prefix hashing**.

37. Interview-Level Explanation

If an interviewer asks:

"Explain your approach."

A strong answer would be:

"I'll use a running prefix sum and a hash map. At each index, let the current prefix sum be S. A subarray ending at this index has sum K if there was a previous prefix sum equal to S-K, because S - previous = K. Therefore, I store the frequency of each prefix sum seen so far and add the frequency of S-K to the answer. I initialize the map with prefix sum zero occurring once to account for subarrays beginning at index zero. This gives O(n) expected time and O(n) space."

That is the level of explanation you should practice.

38. Brute Force → Optimized Thought Process

In interviews, show progression.

Brute force

Try every subarray

Complexity:

$O(n^2)$

with running sums.

Then ask:

Are we repeatedly calculating information that can be reused?

Yes.

Use:

Prefix Sum

Then ask:

How do I find the previous prefix sum I need quickly?

Use:

Hash Map

Final:

$O(n)$

This demonstrates problem-solving rather than memorization.

39. Prefix Sum + Hashing Mental Model

Think of every prefix sum as a **checkpoint**.

Example:

Array:

1	2	-1	3	-2
■	■	■	■	■
↓	↓	↓	↓	↓

Prefix:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 3$

If two checkpoints have the appropriate difference:

```
current prefix
-
old prefix
=
target
```

then everything between those checkpoints is the desired subarray.

The hash map allows us to remember those checkpoints.

40. What You Should Practice Today

For your current Google preparation, don't solve 15 variations.

Master these **three**:

1. Range Sum Query — Immutable

Learn:

Prefix Sum

Goal:

$O(1)$ range query

2. Subarray Sum Equals K

Learn:

Prefix Sum + HashMap

Goal:

count valid subarrays

3. Contiguous Array

Learn:

Transformation
+
Prefix Sum
+
HashMap

Goal:

longest subarray satisfying balance condition

Then revisit them without looking at the solution.

41. Your Day 5 DSA Notes Should Contain

Create a concise revision page containing:

Prefix Sum
↓
Prefix Sum Formula
↓
Subarray Sum Formula
↓
 $\text{current} - \text{previous} = K$
↓
 $\text{previous} = \text{current} - K$
↓
HashMap
↓
Count / Earliest Index / Existence
↓
Negative Numbers
↓
Sliding Window comparison
↓
Modulo variation
↓
Transformation examples

42. Final Pattern Summary

Basic Prefix Sum

$\text{prefix}[i] = \text{sum of elements before } i$

Used for:

range sum queries

Prefix Sum + Hashing

$\text{current_prefix} - \text{previous_prefix} = \text{target}$

Therefore:

$\text{previous_prefix} = \text{current_prefix} - \text{target}$

Use HashMap to find it.

Count

prefix → frequency

Longest

prefix → earliest index

Zero Sum

repeated prefix sum

Divisible by K

prefix % K → frequency

Balanced transformed problem

```
transform values
  ↓
prefix sum
  ↓
repeated prefix state
  ↓
desired subarray
```

43. The one sentence to remember

Whenever a contiguous subarray condition can be expressed as the difference between two cumulative states, use Prefix Sum; whenever you need to find a matching previous state efficiently, add a Hash Map.

This pattern is worth mastering deeply because it connects **arrays** → **hashing** → **mathematical transformation** → **invariants** → **O(n) optimization**, exactly the kind of progression you want in Google DSA interviews.